Table of Contents

AI Portfolio Optimizer

Full-Stack Developer Guide

Build a Hedge Fund-Grade Portfolio Optimizer with Python, FastAPI, React, Monte Carlo Simulation & Claude AI Analysis

Python 3.11 | FastAPI | React 18 | Tailwind CSS | Plotly | Claude API | Monte Carlo | MPT

10,000+	3	8+	100%
Monte Carlo Portfolios	Optimization Targets	Core Modules	Production Ready

WHAT YOU'LL BUILD:

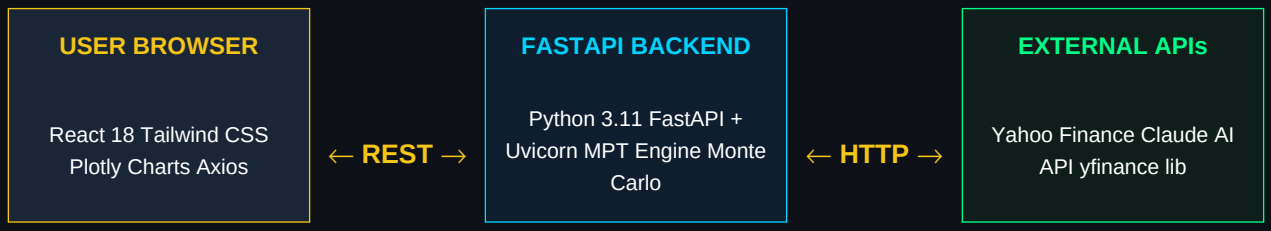
- Efficient Frontier visualization with interactive Plotly charts
- Monte Carlo simulation engine with 10,000 portfolio scenarios
- Claude AI analysis: strengths, risks & diversification scoring

01 Introduction & Architecture Overview

What You're Building

This guide walks you through building a **production-ready AI Portfolio Optimizer** — a full-stack web application that combines Modern Portfolio Theory (MPT), Monte Carlo simulation, and Claude AI to deliver institutional-grade portfolio analysis. The system fetches real market data, runs 10,000+ portfolio simulations, plots the Efficient Frontier, and uses Claude to explain portfolio quality in plain English.

System Architecture



Core Capabilities

Efficient Frontier
Plots 10,000 random portfolios and identifies the optimal risk-return curve using Modern Portfolio Theory.

Max Sharpe & Min Vol
Automatically identifies and highlights the Maximum Sharpe Ratio and Minimum Volatility portfolios.

Claude AI Analysis
Uses Anthropic's Claude API to generate natural language insights about portfolio quality, risks, and improvements.

Real Market Data
Pulls historical OHLCV data from Yahoo Finance using the yfinance Python library — no API key needed.

02 Project Folder Structure

Complete Project Layout

The project follows a clean separation between backend and frontend. The backend is a standalone Python FastAPI service; the frontend is a React application bootstrapped with Vite. Both are containerized with Docker for production deployment.

```
portfolio-optimizer/
├── backend/
│   ├── app/
│   │   ├── __init__.py
│   │   ├── main.py # FastAPI app entry point
│   │   ├── routers/
│   │   │   ├── portfolio.py # /api/optimize endpoint
│   │   │   └── analysis.py # /api/analyze (Claude AI)
│   │   ├── services/
│   │   │   ├── data_fetcher.py # Yahoo Finance data
│   │   │   ├── calculator.py # Returns, vol, Sharpe
│   │   │   ├── optimizer.py # Monte Carlo + MPT
│   │   │   └── ai_analyst.py # Claude API integration
│   │   ├── models/
│   │   │   ├── request.py # Pydantic input models
│   │   │   └── response.py # Pydantic output models
│   │   └── config.py # Settings & env vars
│   ├── tests/
│   │   └── test_optimizer.py
│   ├── requirements.txt
│   └── Dockerfile
├── frontend/
│   ├── src/
│   │   ├── App.tsx
│   │   ├── components/
│   │   │   ├── PortfolioInput.tsx
│   │   │   ├── EfficientFrontier.tsx
│   │   │   ├── AllocationTable.tsx
│   │   │   ├── MetricsCard.tsx
│   │   │   └── AIAnalysis.tsx
│   │   ├── hooks/
│   │   │   └── usePortfolio.ts
│   │   ├── api/
│   │   │   └── portfolioApi.ts
│   │   ├── styles/
│   │   └── globals.css
│   ├── package.json
│   ├── tailwind.config.js
│   └── Dockerfile
└── docker-compose.yml
```

```
■■■ .env.example
■■■ README.md
```

03 Backend: Python + FastAPI Setup

requirements.txt

```
fastapi==0.111.0
uvicorn[standard]==0.30.0
yfinance==0.2.40
pandas==2.2.2
numpy==1.26.4
scipy==1.13.0
anthropic==0.28.0
python-dotenv==1.0.1
pydantic==2.7.1
pydantic-settings==2.3.0
httpx==0.27.0
python-multipart==0.0.9
```

app/config.py — Settings

```
from pydantic_settings import BaseSettings
from functools import lru_cache
class Settings(BaseSettings):
ANTHROPIC_API_KEY: str = ""
CORS_ORIGINS: list[str] = ["http://localhost:5173"]
MAX_TICKERS: int = 20
SIMULATION_RUNS: int = 10_000
DATA_PERIOD: str = '2y' # 2 years of history
RISK_FREE_RATE: float = 0.052 # US 10Y yield
class Config:
env_file = ".env"
@lru_cache()
def get_settings() -> Settings:
return Settings()
```

app/main.py — Application Entry Point

```
from fastapi import FastAPI
from fastapi.middleware.cors import CORSMiddleware
from app.routers import portfolio, analysis
from app.config import get_settings
settings = get_settings()
app = FastAPI(title="AI Portfolio Optimizer",
description="Hedge fund grade portfolio analysis",
version="1.0.0")
```

```
app.add_middleware(
CORSMiddleware,
allow_origins=settings.CORS_ORIGINS,
allow_methods=['*'],
allow_headers=['*'],
)
app.include_router(portfolio.router, prefix="/api")
app.include_router(analysis.router, prefix="/api")
@app.get('/health')
async def health(): return {'status': 'ok'}
```

Pydantic Models — models/request.py

```
from pydantic import BaseModel, Field, validator
from typing import Literal
class PortfolioRequest(BaseModel):
tickers: list[str] = Field(
..., min_items=2, max_items=20,
example=["AAPL","MSFT","GOOGL","AMZN","NVDA"]
)
investment_amount: float = Field(
..., gt=0, example=100_000
)
risk_profile: Literal['conservative','moderate','aggressive']
@validator('tickers', each_item=True)
def uppercase_ticker(cls, v):
return v.strip().upper()
```

04 Financial Calculations Module

services/data_fetcher.py

This module fetches 2 years of daily adjusted closing prices from Yahoo Finance and computes log returns for statistical stability.

```
import yfinance as yf
import pandas as pd
import numpy as np
from app.config import get_settings
settings = get_settings()
def fetch_price_data(tickers: list[str]) -> pd.DataFrame:
"""Download adjusted close prices for all tickers."""
data = yf.download(
tickers=tickers,
period=settings.DATA_PERIOD,
auto_adjust=True,
progress=False
)['Close']
# Drop tickers with >5% missing data
threshold = len(data) * 0.95
data = data.dropna(thresh=int(threshold), axis=1)
data = data.ffill().dropna()
return data
def compute_log_returns(prices: pd.DataFrame) -> pd.DataFrame:
"""Compute daily log returns."""
return np.log(prices / prices.shift(1)).dropna()
```

services/calculator.py — Core Financial Metrics

```
import numpy as np
import pandas as pd
from app.config import get_settings
TRADING_DAYS = 252
settings = get_settings()
def annualized_return(log_returns: pd.DataFrame) -> np.ndarray:
"""Mean daily log return × 252."""
return log_returns.mean().values * TRADING_DAYS
def annualized_volatility(log_returns: pd.DataFrame) -> np.ndarray:
"""Std dev of daily returns × sqrt(252)."""
return log_returns.std().values * np.sqrt(TRADING_DAYS)
def covariance_matrix(log_returns: pd.DataFrame) -> np.ndarray:
"""Annualized covariance matrix."""
return log_returns.cov().values * TRADING_DAYS
def sharpe_ratio(
```

```
    portfolio_return: float,
    portfolio_vol: float,
    risk_free_rate: float = None
    ) -> float:
    """(Rp - Rf) / σp"""
    rf = risk_free_rate or settings.RISK_FREE_RATE
    if portfolio_vol == 0:
    return 0.0
    return (portfolio_return - rf) / portfolio_vol

    def portfolio_performance(
    weights: np.ndarray,
    mean_ret: np.ndarray,
    cov_matrix: np.ndarray,
    ) -> tuple[float, float, float]:
    """Return (annual_return, annual_vol, sharpe) for a weight vector."""
    ret = float(np.dot(weights, mean_ret))
    vol = float(np.sqrt(weights @ cov_matrix @ weights))
    sr = sharpe_ratio(ret, vol)
    return ret, vol, sr
```

05 Monte Carlo Simulation Engine

What is Monte Carlo Portfolio Simulation?

Monte Carlo simulation generates thousands of random portfolio weight combinations to map the risk-return space. Each portfolio is a point on the scatter plot. The outer boundary of these points is the **Efficient Frontier** — portfolios with maximum return for a given level of risk.

services/optimizer.py — Monte Carlo Engine

```
import numpy as np
from dataclasses import dataclass
from app.services.calculator import portfolio_performance
from app.config import get_settings
settings = get_settings()
@dataclass
class SimulationResult:
weights: np.ndarray # (n_sims, n_assets)
returns: np.ndarray # (n_sims,)
vols: np.ndarray # (n_sims,)
sharpes: np.ndarray # (n_sims,)
def run_monte_carlo(
mean_ret: np.ndarray,
cov_matrix: np.ndarray,
n_assets: int,
) -> SimulationResult:
"""Run 10,000 random portfolio simulations."""
n = settings.SIMULATION_RUNS
rng = np.random.default_rng(seed=42)
# Draw random weights and normalize to sum to 1
raw_w = rng.random((n, n_assets))
weights = raw_w / raw_w.sum(axis=1, keepdims=True)
returns = np.zeros(n)
vols = np.zeros(n)
sharpes = np.zeros(n)
for i in range(n):
ret, vol, sr = portfolio_performance(
weights[i], mean_ret, cov_matrix
)
returns[i], vols[i], sharpes[i] = ret, vol, sr
return SimulationResult(weights, returns, vols, sharpes)
def find_max_sharpe(result: SimulationResult) -> int:
"""Index of portfolio with maximum Sharpe ratio."""
return int(np.argmax(result.sharpes))
def find_min_vol(result: SimulationResult) -> int:
"""Index of portfolio with minimum volatility."""
```

```
    return int(np.argmin(result.vols))
```

06 Portfolio Optimization Logic

Scipy-Based Optimization (Optional Enhancement)

While Monte Carlo gives us a great visual, we can also use **scipy.optimize.minimize** to find the mathematically exact Maximum Sharpe and Minimum Volatility portfolios using Sequential Least Squares Programming (SLSQP).

```
import numpy as np
from scipy.optimize import minimize
from app.services.calculator import portfolio_performance
def optimize_max_sharpe(
mean_ret: np.ndarray,
cov_matrix: np.ndarray,
n_assets: int,
) -> np.ndarray:
"""Find exact max-Sharpe weights using SLSQP."""
def neg_sharpe(w):
r, v, sr = portfolio_performance(w, mean_ret, cov_matrix)
return -sr # minimize negative = maximize Sharpe
constraints = [{'type': 'eq', 'fun': lambda w: np.sum(w) - 1}]
bounds = [(0.01, 0.40)] * n_assets # 1%-40% per asset
x0 = np.ones(n_assets) / n_assets # equal weight start
result = minimize(
neg_sharpe, x0, method='SLSQP',
bounds=bounds, constraints=constraints,
options={'maxiter': 1000, 'ftol': 1e-9}
)
return result.x
def optimize_min_volatility(
cov_matrix: np.ndarray,
n_assets: int,
) -> np.ndarray:
"""Find exact minimum variance portfolio weights."""
def portfolio_vol(w):
return float(np.sqrt(w @ cov_matrix @ w))
constraints = [{'type': 'eq', 'fun': lambda w: np.sum(w) - 1}]
bounds = [(0.01, 0.40)] * n_assets
x0 = np.ones(n_assets) / n_assets
result = minimize(
portfolio_vol, x0, method='SLSQP',
bounds=bounds, constraints=constraints,
options={'maxiter': 1000, 'ftol': 1e-9}
)
return result.x
```

Risk Profile Constraints

The user's risk profile modifies position limits to ensure the optimizer respects their investment goals:

Risk Profile	Max Single Position	Optimization Target	Typical Sharpe Range
Conservative	25%	Min Volatility	0.4 – 0.8
Moderate	35%	Max Sharpe	0.8 – 1.2
Aggressive	50%	Max Return	1.0 – 1.8

07 FastAPI Routes & Endpoints

routers/portfolio.py — Main Optimization Endpoint

```
from fastapi import APIRouter, HTTPException
from app.models.request import PortfolioRequest
from app.models.response import PortfolioResponse
from app.services.data_fetcher import fetch_price_data, compute_log_returns
from app.services.calculator import (annualized_return, annualized_volatility,
covariance_matrix, portfolio_performance)
from app.services.optimizer import (run_monte_carlo, find_max_sharpe,
find_min_vol)
router = APIRouter()
@router.post('/optimize', response_model=PortfolioResponse)
async def optimize_portfolio(req: PortfolioRequest):
try:
# 1. Fetch market data
prices = fetch_price_data(req.tickers)
returns = compute_log_returns(prices)
# 2. Compute statistics
mean_ret = annualized_return(returns)
cov_matrix = covariance_matrix(returns)
tickers = list(prices.columns)
# 3. Run Monte Carlo
sim = run_monte_carlo(mean_ret, cov_matrix, len(tickers))
# 4. Find optimal portfolios
ms_idx = find_max_sharpe(sim)
mv_idx = find_min_vol(sim)
def make_allocation(idx):
w = sim.weights[idx]
r, v, s = portfolio_performance(w, mean_ret, cov_matrix)
dollar = {t: float(w[i] * req.investment_amount)
for i, t in enumerate(tickers)}
return {
'weights': dict(zip(tickers, w.round(4).tolist())),
'dollar_alloc': dollar,
'annual_return': round(r * 100, 2),
'annual_vol': round(v * 100, 2),
'sharpe_ratio': round(s, 3),
}
return PortfolioResponse(
tickers = tickers,
max_sharpe = make_allocation(ms_idx),
min_volatility = make_allocation(mv_idx),
simulation_points = {
'returns': sim.returns.tolist(),
'vols': sim.vols.tolist(),
```

```
    'sharpes': sim.sharpes.tolist(),
    },
    individual_stats = {
    t: {
    'annual_return': round(float(mean_ret[i]) * 100, 2),
    'annual_vol': round(float(annualized_volatility(returns)[i]) * 100, 2),
    }
    for i, t in enumerate(tickers)
    }
    )
    except Exception as e:
    raise HTTPException(status_code=500, detail=str(e))
```

08 Frontend: React + Tailwind Setup

Initialize Vite + React + TypeScript

```
# Create Vite project
npm create vite@latest frontend -- --template react-ts
cd frontend
# Install dependencies
npm install
npm install -D tailwindcss postcss autoprefixer
npm install axios plotly.js react-plotly.js
npm install @types/plotly.js
npm install lucide-react clsx
# Init Tailwind
npx tailwindcss init -p
```

tailwind.config.js — Custom Theme

```
/** @type {import('tailwindcss').Config} */
export default {
content: ['./index.html', './src/**/*.{ts,tsx}'],
theme: {
extend: {
colors: {
navy: { DEFAULT: '#0A0E1A', 900: '#060810', 800: '#0D1117' },
gold: { DEFAULT: '#F5C518', light: '#FFD700', dark: '#C8960C' },
cyan: { DEFAULT: '#00D4FF', dark: '#0099CC' },
success: '#00FF88',
danger: '#FF4444',
card: '#161B27',
border: '#1E2D3D',
},
fontFamily: {
mono: ['JetBrains Mono', 'Fira Code', 'monospace'],
},
backgroundImage: {
'gradient-radial': 'radial-gradient(var(--tw-gradient-stops))',
},
},
},
plugins: [],
}
```

api/portfolioApi.ts — Axios Client

```
import axios from 'axios';

const API_BASE = import.meta.env.VITE_API_URL ?? 'http://localhost:8000';

const client = axios.create({
baseURL: `${API_BASE}/api`,
headers: { 'Content-Type': 'application/json' },
});

export interface PortfolioRequest {
tickers: string[];
investment_amount: number;
risk_profile: 'conservative' | 'moderate' | 'aggressive';
}

export const optimizePortfolio = (req: PortfolioRequest) =>
client.post('/optimize', req).then(r => r.data);

export const analyzePortfolio = (data: any) =>

client.post('/analyze', data).then(r => r.data);
```

09 Portfolio Input Component

components/PortfolioInput.tsx

```
import { useState } from 'react';
import { Search, Plus, X, TrendingUp } from 'lucide-react';
import clsx from 'clsx';
const RISK_PROFILES = [
{ id: 'conservative', label: 'Conservative', color: 'text-cyan-400',
desc: 'Capital preservation, lower volatility' },
{ id: 'moderate', label: 'Moderate', color: 'text-gold',
desc: 'Balanced risk-return tradeoff' },
{ id: 'aggressive', label: 'Aggressive', color: 'text-success',
desc: 'Maximum growth, higher risk tolerance' },
];
export function PortfolioInput({ onSubmit, loading }) {
const [tickers, setTickers] = useState<string[]>(['AAPL','MSFT','NVDA']);
const [input, setInput] = useState('');
const [amount, setAmount] = useState(100000);
const [profile, setProfile] = useState('moderate');
const addTicker = () => {
const t = input.trim().toUpperCase();
if (t && !tickers.includes(t) && tickers.length < 20) {
setTickers(prev => [...prev, t]);
setInput('');
}
};
return (
<div className='bg-card border border-border rounded-xl p-6'>
<h2 className='text-gold font-bold text-xl mb-6 flex items-center gap-2'>
<TrendingUp size={20}/> Portfolio Configuration
</h2>
{/* Ticker Input */}
<div className='flex gap-2 mb-3'>
<input
value={input} onChange={e => setInput(e.target.value.toUpperCase())}
onKeyDown={e => e.key === 'Enter' && addTicker()}
placeholder='Enter ticker (e.g. AAPL)'
className='flex-1 bg-navy-800 border border-border rounded-lg
px-4 py-2.5 text-white text-sm font-mono
focus:ring-1 focus:ring-gold outline-none'
/>
<button onClick={addTicker}
className='bg-gold text-navy px-4 py-2.5 rounded-lg font-bold
hover:bg-gold-light transition-colors'>
<Plus size={18}/>
</button>
</div>
```

```
{/* Ticker Chips */}
<div className='flex flex-wrap gap-2 mb-5'>
{tickers.map(t => (
<span key={t} className='flex items-center gap-1.5
bg-navy-900 border border-border rounded-full
px-3 py-1 text-sm font-mono text-cyan-400'>
{t}
<button onClick={() => setTickers(p => p.filter(x => x !== t))}
className='text-gray-500 hover:text-red-400'>
<X size={12}/>
</button>
</span>
))}
</div>
{/* Investment Amount */}
<div className='mb-5'>
<label className='text-gold text-xs font-bold uppercase tracking-wider mb-2 block'>
Investment Amount (USD)
</label>
<input type='number' value={amount}
onChange={e => setAmount(Number(e.target.value))}
className='w-full bg-navy-800 border border-border rounded-lg
px-4 py-2.5 text-white font-mono outline-none
focus:ring-1 focus:ring-gold'/>
</div>
{/* Risk Profile */}
<div className='grid grid-cols-3 gap-3 mb-6'>
{RISK_PROFILES.map(rp => (
<button key={rp.id} onClick={() => setProfile(rp.id)}
className={clsx(
'border rounded-lg p-3 text-left transition-all',
profile === rp.id
? 'border-gold bg-gold/10'
: 'border-border bg-navy-800 hover:border-gold/50'
)}>
<div className={clsx('font-bold text-sm', rp.color)}>
{rp.label}</div>
<div className='text-gray-500 text-xs mt-1'>{rp.desc}</div>
</button>
))}
</div>
<button onClick={() =>
onSubmit({tickers,investment_amount:amount,risk_profile:profile})}
disabled={loading || tickers.length < 2}
className='w-full bg-gold hover:bg-gold-light disabled:opacity-50
text-navy font-bold py-3 rounded-lg transition-colors'>
{loading ? 'Optimizing...' : 'Run Optimization ■'}
</button>
</div>
);
```

```
    }
```

10 Efficient Frontier Chart (Plotly)

components/EfficientFrontier.tsx

This component renders the Monte Carlo scatter plot with color-coded Sharpe ratios, and highlights the Maximum Sharpe and Minimum Volatility portfolios as special markers.

```
import Plot from 'react-plotly.js';

export function EfficientFrontier({ simData, maxSharpe, minVol }) {
const { returns, vols, sharpes } = simData;

const traces = [
// Monte Carlo scatter — color by Sharpe ratio
{
type: 'scatter', mode: 'markers',
x: vols.map((v: number) => v * 100),
y: returns.map((r: number) => r * 100),
marker: {
color: sharpes, colorscale: 'Viridis',
size: 3, opacity: 0.6,
colorbar: { title: 'Sharpe Ratio', thickness: 12,
tickfont: { color: '#8B9DB0' },
titlefont: { color: '#F5C518' } }
},
name: 'Monte Carlo Portfolios',
hovertemplate:
'<b>Return:</b> %{y:.1f}%<br>' +
'<b>Risk:</b> %{x:.1f}%<extra></extra>'
},
// Max Sharpe star
{
type: 'scatter', mode: 'markers+text',
x: [maxSharpe.annual_vol],
y: [maxSharpe.annual_return],
marker: { symbol: 'star', size: 18, color: '#F5C518' },
text: [' Max Sharpe'], textfont: { color: '#F5C518', size: 11 },
name: 'Max Sharpe',
hovertemplate:
'<b>Max Sharpe Portfolio</b><br>' +
'Return: %{y:.1f}%<br>Risk: %{x:.1f}%<extra></extra>'
},
// Min Vol diamond
{
type: 'scatter', mode: 'markers+text',
x: [minVol.annual_vol],
y: [minVol.annual_return],
marker: { symbol: 'diamond', size: 16, color: '#00D4FF' },
text: [' Min Volatility'], textfont: { color: '#00D4FF', size: 11 },
name: 'Min Volatility',
```

```
},
];
const layout = {
paper_bgcolor: '#0D1117',
plot_bgcolor: '#0D1117',
title: { text: 'Efficient Frontier', font: { color: '#F5C518', size: 16 } },
xaxis: { title: 'Annualized Risk (%)', gridcolor: '#1E2D3D',
color: '#8B9DB0', tickfont: { color: '#8B9DB0' } },
yaxis: { title: 'Annualized Return (%)', gridcolor: '#1E2D3D',
color: '#8B9DB0', tickfont: { color: '#8B9DB0' } },
legend: { font: { color: '#8B9DB0' }, bgcolor: 'rgba(0,0,0,0)' },
margin: { t: 50, b: 50, l: 60, r: 20 },
};
return (
<div className='bg-card border border-border rounded-xl p-4'>
<Plot data={traces} layout={layout}
style={{ width: '100%', height: '480px' }}
config={{ displayModeBar: false, responsive: true }}/>
</div>
);
}
```

11 Claude AI Analysis Integration

services/ai_analyst.py — Backend AI Service

```
import anthropic
from app.config import get_settings
settings = get_settings()
MODEL = "claude-opus-4-5"
def build_analysis_prompt(portfolio_data: dict) -> str:
ms = portfolio_data['max_sharpe']
mv = portfolio_data['min_volatility']
tickers = portfolio_data['tickers']
ind_stats = portfolio_data.get('individual_stats', {})
return f'''
You are a senior quantitative portfolio analyst at a top hedge fund.
Analyze this portfolio optimization result and provide institutional-grade insights.
## Portfolio Overview
Tickers: {', '.join(tickers)}
Investment Amount: ${portfolio_data.get('investment_amount', 'N/A'):,}
Risk Profile: {portfolio_data.get('risk_profile', 'moderate').title()}
## Maximum Sharpe Portfolio
- Weights: {ms['weights']}
- Expected Annual Return: {ms['annual_return']}%
- Expected Annual Volatility: {ms['annual_vol']}%
- Sharpe Ratio: {ms['sharpe_ratio']}
## Minimum Volatility Portfolio
- Weights: {mv['weights']}
- Expected Annual Return: {mv['annual_return']}%
- Expected Annual Volatility: {mv['annual_vol']}%
- Sharpe Ratio: {mv['sharpe_ratio']}
## Individual Asset Statistics
{ind_stats}
Please provide a structured analysis covering:
1. **Portfolio Strengths** — What makes these allocations effective?
2. **Concentration Risks** — Identify sector or single-stock concentration issues.
3. **Diversification Quality** — How well-diversified is this portfolio?
4. **Suggested Improvements** — Specific actionable recommendations.
5. **Market Context** — How might current macro conditions affect these holdings?
Format your response as clean, professional markdown with clear headers.
Be specific, data-driven, and concise. No generic disclaimers.'''
async def analyze_portfolio(portfolio_data: dict) -> str:
client = anthropic.Anthropic(api_key=settings.ANTHROPIC_API_KEY)
prompt = build_analysis_prompt(portfolio_data)
message = client.messages.create(
model=MODEL,
max_tokens=1500,
messages=[{'role': 'user', 'content': prompt}]
```

```
)
return message.content[0].text
```

components/AIAnalysis.tsx — Frontend Display

```
import { useState } from 'react';
import { Brain, ChevronDown, ChevronUp, Sparkles } from 'lucide-react';
import { analyzePortfolio } from '../api/portfolioApi';
export function AIAnalysis({ portfolioData }) {
const [analysis, setAnalysis] = useState<string | null>(null);
const [loading, setLoading] = useState(false);
const [expanded, setExpanded] = useState(true);
const runAnalysis = async () => {
setLoading(true);
try {
const res = await analyzePortfolio(portfolioData);
setAnalysis(res.analysis);
} finally { setLoading(false); }
};
return (
<div className='bg-card border border-border rounded-xl overflow-hidden'>
<div className='flex items-center justify-between p-5
border-b border-border bg-navy-900/50'>
<div className='flex items-center gap-2'>
<Brain className='text-gold' size={20}/>
<h3 className='text-gold font-bold text-lg'>
Claude AI Portfolio Analysis</h3>
<span className='text-xs text-gray-500 font-mono ml-2'>
Powered by claude-opus-4-5</span>
</div>
<button onClick={() => setExpanded(!expanded)}
className='text-gray-500 hover:text-white transition-colors'>
{expanded ? <ChevronUp size={18}/> : <ChevronDown size={18}/>}
</button>
</div>
{expanded && (
<div className='p-5'>
{!analysis ? (
<div className='text-center py-8'>
<Sparkles className='mx-auto text-gold mb-3' size={32}/>
<p className='text-gray-400 text-sm mb-4'>
Get AI-powered insights on your portfolio</p>
<button onClick={runAnalysis} disabled={loading}
className='bg-gold/10 border border-gold text-gold
px-6 py-2.5 rounded-lg font-bold text-sm
hover:bg-gold/20 transition-colors disabled:opacity-50'>
{loading ? 'Analyzing with Claude...' : 'Run AI Analysis'}
</button>
```

```
</div>
) : (
<div className='prose prose-invert prose-gold max-w-none text-sm
text-gray-300 leading-relaxed'>
<ReactMarkdown>{analysis}</ReactMarkdown>
</div>
)}
</div>
)}
</div>
);
}
```

● 12 Dashboard Layout & Styling

App.tsx — Main Dashboard Layout

```
import { useState } from 'react';
import { PortfolioInput } from './components/PortfolioInput';
import { EfficientFrontier } from './components/EfficientFrontier';
import { AllocationTable } from './components/AllocationTable';
import { MetricsCard } from './components/MetricsCard';
import { AIAnalysis } from './components/AIAnalysis';
import { optimizePortfolio } from './api/portfolioApi';
export default function App() {
const [result, setResult] = useState(null);
const [loading, setLoading] = useState(false);
const [error, setError] = useState('');
const handleOptimize = async (req) => {
setLoading(true); setError('');
try {
const data = await optimizePortfolio(req);
setResult({ ...data, ...req });
} catch(e) {
setError(e.response?.data?.detail ?? 'Optimization failed');
} finally { setLoading(false); }
};
return (
<div className='min-h-screen bg-navy text-white font-mono'>
{/* Navbar */}
<nav className='border-b border-border bg-navy-900/80 backdrop-blur sticky top-0 z-50'>
<div className='max-w-7xl mx-auto px-6 h-14 flex items-center justify-between'>
<span className='text-gold font-bold tracking-wider'>
■ AI PORTFOLIO OPTIMIZER</span>
<span className='text-xs text-gray-500'>
Powered by Claude AI + MPT</span>
</div>
</nav>
<main className='max-w-7xl mx-auto px-6 py-8'>
{/* Top: Input + Metrics */}
<div className='grid grid-cols-12 gap-6 mb-6'>
<div className='col-span-4'>
<PortfolioInput onSubmit={handleOptimize} loading={loading}/>
</div>
<div className='col-span-8'>
{result && (
<div className='grid grid-cols-2 gap-4'>
<MetricsCard title='Max Sharpe' data={result.max_sharpe}
accent='gold'/>
<MetricsCard title='Min Volatility' data={result.min_volatility}
accent='cyan'/>
```

```
</div>
)}
</div>
</div>
{/* Chart */}
{result && (
<>
<div className='mb-6'>
<EfficientFrontier
simData={result.simulation_points}
maxSharpe={result.max_sharpe}
minVol={result.min_volatility}/>
</div>
{/* Allocations */}
<div className='grid grid-cols-2 gap-6 mb-6'>
<AllocationTable
title='Max Sharpe Allocation'
data={result.max_sharpe} accent='gold'/>
<AllocationTable
title='Min Vol Allocation'
data={result.min_volatility} accent='cyan'/>
</div>
{/* AI Analysis */}
<AIAnalysis portfolioData={result}/>
</>
)}
</main>
</div>
);
}
```

● 13 Environment Variables & Config

.env.example — Root Config File

```
# ■■ Backend ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
ANTHROPIC_API_KEY=sk-ant-api03-xxxxxxxxxxxxxxxxxxxxxxx
CORS_ORIGINS=["http://localhost:5173","http://localhost:3000"]
SIMULATION_RUNS=10000
RISK_FREE_RATE=0.052
DATA_PERIOD=2y
# ■■ Frontend ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
VITE_API_URL=http://localhost:8000
# ■■ Docker ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
BACKEND_PORT=8000
FRONTEND_PORT=5173
```

■ Security Warning: Never commit your .env file to version control. Add it to .gitignore immediately. Rotate your Anthropic API key if accidentally exposed.

14 Docker & Deployment Guide

docker-compose.yml

```
version: '3.9'

services:
backend:
build: ./backend
ports: ['8000:8000']
env_file: .env
volumes: ['./backend:/app']
command: uvicorn app.main:app --host 0.0.0.0 --port 8000 --reload
healthcheck:
test: ['CMD', 'curl', '-f', 'http://localhost:8000/health']
interval: 30s
timeout: 10s
retries: 3
frontend:
build: ./frontend
ports: ['5173:5173']
env_file: .env
volumes: ['./frontend:/app', '/app/node_modules']
depends_on: [backend]
environment:
- VITE_API_URL=http://backend:8000
```

backend/Dockerfile

```
FROM python:3.11-slim

WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY . .
EXPOSE 8000
CMD ["uvicorn", "app.main:app", "--host", "0.0.0.0", "--port", "8000"]
```

Quick Start Commands

```
# Clone & setup
git clone https://github.com/yourorg/portfolio-optimizer.git
cd portfolio-optimizer
cp .env.example .env
# Edit .env and add your ANTHROPIC_API_KEY
# Run with Docker (recommended)
```

```
docker-compose up --build

# Run locally (development)
cd backend && pip install -r requirements.txt
uvicorn app.main:app --reload
cd frontend && npm install && npm run dev

# Access the application
# Frontend: http://localhost:5173
# API docs: http://localhost:8000/docs
```

Production Deployment (AWS / GCP / Railway)

Platform	Backend Command	Notes
Railway	`Auto-detected from Dockerfile`	Add env vars in dashboard
Render.com	`uvicorn app.main:app --port $PORT`	Free tier available
AWS ECS	`docker-compose push + ecs-cli`	Use ECR for image storage
Vercel (FE)	`npm run build`	Set VITE_API_URL in env

● 15 Testing & Validation

tests/test_optimizer.py

```
import pytest, numpy as np
from fastapi.testclient import TestClient
from app.main import app
from app.services.calculator import portfolio_performance, sharpe_ratio
from app.services.optimizer import run_monte_carlo, find_max_sharpe
client = TestClient(app)
# ■■ Unit Tests ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
def test_sharpe_ratio():
sr = sharpe_ratio(0.15, 0.20, 0.05)
assert abs(sr - 0.50) < 0.001
def test_portfolio_performance():
weights = np.array([0.5, 0.5])
mean_ret = np.array([0.12, 0.10])
cov = np.array([[0.04, 0.01], [0.01, 0.03]])
ret, vol, sr = portfolio_performance(weights, mean_ret, cov)
assert 0 < ret < 1
assert 0 < vol < 1
assert sr > 0
def test_monte_carlo_shapes():
mean_ret = np.array([0.12, 0.10, 0.15])
cov = np.eye(3) * 0.04
result = run_monte_carlo(mean_ret, cov, n_assets=3)
assert result.weights.shape == (10_000, 3)
assert result.returns.shape == (10_000,)
# Weights sum to 1
np.testing.assert_allclose(result.weights.sum(axis=1), 1, atol=1e-10)
# ■■ Integration Tests ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
def test_health():
r = client.get('/health')
assert r.status_code == 200
def test_optimize_endpoint():
payload = {
"tickers": ["AAPL", "MSFT", "GOOGL"],
"investment_amount": 10000,
"risk_profile": "moderate"
}
r = client.post("/api/optimize", json=payload)
assert r.status_code == 200
data = r.json()
assert "max_sharpe" in data
assert "min_volatility" in data
assert "simulation_points" in data
assert data['max_sharpe']['sharpe_ratio'] > 0
```

```
# ■■ Run all tests ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
# pytest tests/ -v --tb=short
```

Summary & Next Steps

Backend Complete
FastAPI + yfinance + Monte Carlo + Scipy optimizer + Claude AI analysis all wired up.

Frontend Complete
React + Tailwind + Plotly Efficient Frontier chart + AI analysis panel.

Production Ready
Docker Compose for local dev + Railway/Render deployment guide included.

Enhancement Ideas
Add user auth, save portfolios to PostgreSQL, add real-time price streaming via WebSocket.

More Charts to Add
Correlation heatmap, rolling Sharpe chart, drawdown analysis, sector exposure pie.

Extend AI Analysis
Add Claude streaming, chart summarization, and voice-over export for Instagram reels.

DEEPTHINKSFINANCE @deepthinksfinance | AI × Finance × Quant	If this guide helped you, follow **@deepthinksfinance** for more AI × Finance content